The second thing that Ruby does with double-quoted strings is expression interpolation. Within the string, the sequence #{EXPRESSION} is replaced by the value of EXPRESSION. We could use this to rewrite our previous method:

```
def say_hello_goodbye(name)
  result = "I don't know why you say goodbye, #{name}, I say hello"
  return result
end
puts say_hello_goodbye("Ringo")
```

produces:

```
I don't know why you say goodbye, Ringo, I say hello
```

When Ruby constructs this string object, it looks at the current value of name and substitutes it into the string. Arbitrarily complex expressions are allowed in the #{...} construct. In the following example, we invoke the capitalize method, defined for all strings, to output our parameter with a leading uppercase letter:

```
def say_hello_goodbye(name)
  result = "I don't know why you say goodbye,  #{name.capitalize}, I say hello"
  return result
end
puts say_hello_goodbye("john")
```

produces:

```
I don't know why you say goodbye,  John, I say hello
```

For more information on strings, as well the other Ruby standard types, see Chapter 7, Standard Types, on page 117.

We could simplify our say_hello_goodbye method some more. In the absence of an explicit return statement, the value returned by a Ruby method is the value of the last expression evaluated, so we can get rid of the temporary variable and the return statement altogether.

```
def say_hello_goodbye(name)
  "I don't know why you say goodbye, #{name}, I say hello"
end
puts say_hello_goodbye("Paul")
```

produces:

```
I don't know why you say goodbye, Paul, I say hello
```

This version is considered more *idiomatic*, by which we mean that it is more in line with how expert Ruby programmers have chosen to write Ruby programs. Idiomatic Ruby tends to lean into Ruby's shortcuts and specific syntax. A good clearinghouse for the guidelines for idiomatic Ruby style can be found in the documentation for the Standard gem at https://github.com/testdouble/standard, which has been used for the code in this book (except where we deliberately break its rules to make a point.)

We promised that this section would be brief. We have just one more topic to cover: Ruby names. For brevity, we'll be using some terms (such as *class variable*) that we aren't going to define here. However, by talking about the rules now, you'll be ahead of the game when we actually come to discuss class variables and the like later.

Ruby uses a convention that may seem strange at first: the first characters of a name indicate how broadly the variable is visible. Local variables, method parameters, and method names